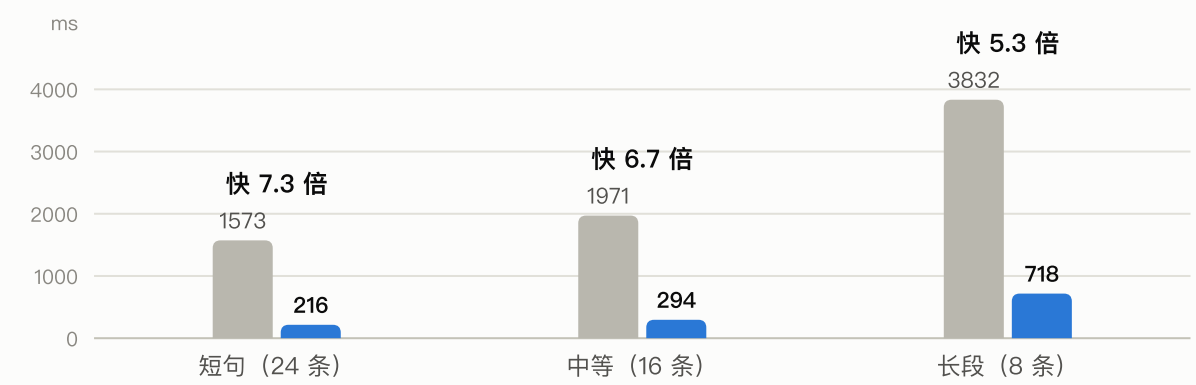
SoL-Pi-Swarm 自动优化 Voxt 本地润色，延迟加速 6.76 倍

同一模型（Qwen3.5-4B-OptiQ-4bit）、同一台 M4 Pro、同样的提示词和解码规则（贪心加重复惩罚）。基线为 Voxt 现在的做法：mlx-swift-lm，每次请求新建会话。延迟从发出请求算到收到完整润色结果。

三组输入的润色延迟降到 216–718 ms

润色延迟：从把识别文字发给模型，到拿到完整润色结果的时间，取中位数，越低越好。



加权加速比（短句 0.5、中等 0.35、长段 0.15 的加权几何平均）

6.76 倍

p90 延迟三组降到 0.36–1.12 s（Voxt 现状 1.69–5.04 s）。系统提示词不再重复计算，首个字出现时间缩短到原来的约十分之一。

首个字出现

116–280 ms

Voxt 现状为 1335–1477 ms

与原版输出完全一致

378 / 384 条

其余 6 条只在两可的并列处分歧

概率分布差异（KL 平均）

0.00006

合法噪声同为 0.00006，门槛 0.0005

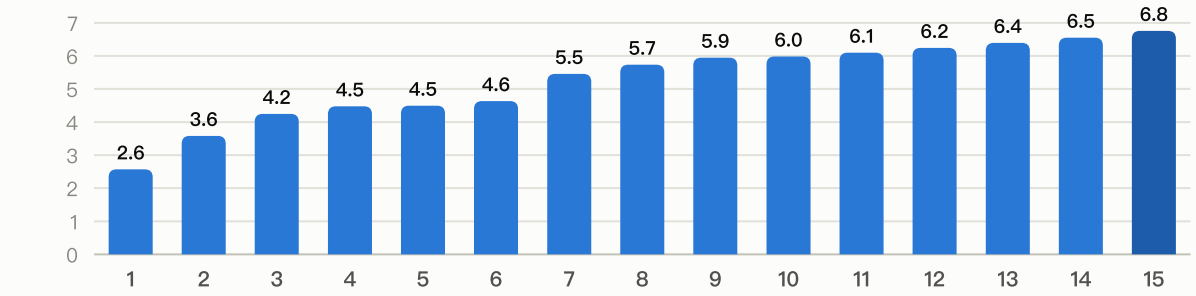
峰值内存

5.91 GB

上限 6.2 GB，Voxt 现状 5.64 GB

24 小时内 15 个有效版本，加速比从 2.57 倍提高到 6.76 倍

横轴为版本号，纵轴为加权加速比，Voxt 现状为 1 倍。每个版本都经过完整评分：384 条正确性比对加 48 条测速输入。



优化方法

- **系统提示词只算一次。** 每次请求都带同一段约 1000 token 的润色说明，原来每次重算要 1.3 s。现在把算完的状态（注意力缓存和线性注意力状态）存下来，后续请求只预填充用户那句话。
- **从输入里拿草稿的投机解码。** 润色后的文字大部分照抄输入，就从输入里找接下来的几个 token 当草稿，模型一次核验一批，猜对的直接收下。接受规则与逐个生成相同（贪心加重复惩罚），结果不变。
- **缓存和草稿利用文本结构。** 缓存延长到用户消息的固定开头；第一轮草稿按位置从输入里取，草稿长度按命中率和核验成本调整。
- **计算更快、少做无用功。** 线性注意力层用多行计算核心，核验编译成固定计算图，9 行和 12–16 行矩阵乘法用 16 行分块，预填充只算最后一行的词表输出；草稿部分被接受时直接截断状态，不再回滚重算。

说明

1. 速度在 48 条测速输入上测，每条跑两遍取平均；正确性在全部 384 条输入上与原版逐条比对，没有一条被判为算错。
2. 第 15 版峰值内存 5.91 GB，比 Voxt 现状高 5%，在 1.1 倍上限内；降低内存和 GPU 占用留到后续任务。
3. 测试文本取自 ASR 语料的识别结果，一半插入了口语填充词；真实听写更口语化，提速幅度可能不同。

评测与过程

正确性门槛先用合法数值噪声和两种植入错误（8 位 KV 缓存、去掉重复惩罚）校准，再冻结。优化由 SoL-Pi-Swarm 完成：Opus 5.5 协调，4 个 DeepSeek V4.1 Flash 工作者，共运行 24 小时（中途暂停约 50 小时，11-02 06:48 结束），费用 $37.46，结束审计通过。